UNIDAD 5 - Inteligencia Colectiva 1

Autómatas Celulares

→ Estructura que tiene entradas **X**, salidas **Y**, un conjunto de estados **E** y un conjunto de reglas **D** que le permiten ir cambiando de estado.

$$A = < X, Y, E, D >$$

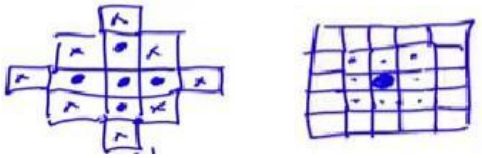
→ Extensión con funciones de salida: $y = \lambda(x, E)$

→ Los estados con reglas de transición se representan como **grafos**.

→ Las **reglas de transición** pueden ser determinísticas y probabilísticas.

Autómatas celulares

- Definición: **R = < A, T, C >** **(A = autómatas que lo componen)**
  (T = topología)
  (C = conectividad)
- Topologías: triangular, rectangular, hexagonal,...
- Acoplamiento:
  - Tipos y tamaños de vecindad: Von Neumann, Moore,...



(radio 2)

  - Tipos de conexiones: isotrópicas, anisotrópicas,...
    Si son todas iguales en todas las direcciones o tienen un sentido preferencial.

Ejemplos:
Juego de la vida de Conway: reglas básicas

- Vivo con menos de 2 vivos en el entorno → muere
- Vivo con más de 3 vivos en el entorno → muere
- Vivo con 2 o 3 vivos en el entorno → vive
- Muerto con 3 vivos en el entorno → nace

→ Mirek's Cellebration
Crecimiento de plantas, bacterias,...
Poblaciones: colonias de hormigas, enjambres de abejas, modelos presa predador,...
Tejidos biológicos: cardíaco, nervioso,...
Fluidos

Agentes

Un *agente* es un sistema que...

- está situado en un *ambiente*
- es capaz de realizar *acciones automáticas*
- para cumplir sus *objetivos* de diseño

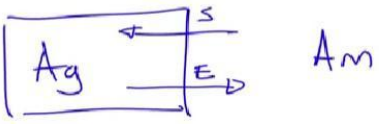
Un *agente* inteligente debe...

- ser proactivo (realizar acciones sin un requerimiento específico)
- ser reactivo (reaccionar ante una demanda o un cambio en el ambiente)

- y poseer habilidad social (poder interactuar con otros agentes)

Un *agente* es todo aquello que...

- percibe su ambiente mediante *sensores*
- y responde o actúa sobre el ambiente mediante *efectores*



Todo agente debe poseer *autonomía*, es decir, capacidad de...

- ***aprender*** de la experiencia
- modificar comportamiento en tiempo de ejecución

Un agente *racional* es aquel que...

- realiza acciones correctas

Un *agente racional* ideal debe ser capaz de...

- percibir
- conocer
- decidir
- actuar

Sistemas multi-agente

Son cooperativos

- por interacción
- por contratos
- por negociación

Un agente posee racionalidad social...

- si puede realizar acciones que generan un beneficio a *todos*
- si ese beneficio es más grande que las pérdidas:

$$\text{utilidad esperada} = f(\text{utilidad individual}) + f(\text{utilidad social})$$

Inteligencia Colectiva - Generalidades

Denominaciones y relaciones:

- Computación evolutiva
- Inteligencia de colonias
- Inteligencia de enjambres
- Inteligencia colaborativa
- Inteligencia social
- ...

Características generales:

- **Auto-organización** (relación con SOM...)
- **Estigmergia**: “colaboración a través del medio físico”
- **Comportamiento emergente**: (comportamiento que uno no define de antemano)

  → inteligencia distribuida, da robustez al sistema
- **Fuerte interacción local** (vecindades)

- **Organización social altamente estructurada** (con roles y jerarquías)
- **Colaboración versus competencia** (siempre hay un compromiso entre colaborar con los demás hacia un fin común o competir y lograr cada uno lo suyo)
- **Componentes estocásticas** (aparecen mucho y son importantes)
- **Bio-inspiración** (casos inspirados en la naturaleza para resolver otros problemas)

Ejemplos de inteligencia colectiva:

• Bandadas de pájaros
• Colonias de hormigas
• Paneles/enjambres de abejas
• Cardúmenes de peces
• Rebaños de ovejas o cabras
• Manadas de predadores

Elementos individuales:

- "Boids": partículas, objetos, elementos...
- Autómatas: redes de autómatas celulares
- Agentes: en estructuras de multi-agentes
- Neuronas…

Algoritmos de inteligencia colectiva:

- ➢ Algoritmos evolutivos
- ➢ Colonias de hormigas
- ➢ Enjambre de partículas
  - • Difusión estocástica
  - • Formación de ríos
  - • Búsqueda gravitacional
  - • Sistema inmune artificial
  - • Algorítmos meméticos
  - • …

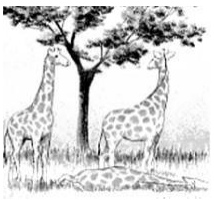
Principales aplicaciones:

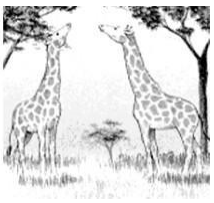
• Optimización: aproximación de funciones, entrenamiento, estimación, identificación, planificación,...
• Búsqueda, ruteo
• Agrupamiento no supervisado, clasificación
• …

Computación Evolutiva: Algoritmos Genéticos

Hace 150 años...
La idea de la evolución genera un cambio de paradigmas tan grande que hasta hoy, incluso en computación, estamos hablando de **Charles R. Darwin**.

Variación y selección natural: si hay variabilidad en la longitud del cuello de las jirafas, las de cuello corto tendrán menos probabilidades de sobrevivir y procrear. Así…





...en la próxima generación habrá menos jirafas de cuello corto.

Y...
¿Qué tiene que ver todo esto con la computación?
¿Y con la "inteligencia" computacional?
¿Podremos ver las ideas de Darwin como un algoritmo? → SÍ
¿Podremos usar estas ideas para resolver problemas con la computadora?

La evolución como un algoritmo

- Poblaciones versus individuos
- Mejores versus "adaptados" → La evolución busca la mejor adaptación a las condiciones actuales, no al mejor "super individuo".
- Aleatoriedad en la selección natural → Los mejores adaptados no tienen asegurada la supervivencia, y viceversa.
- Diversidad → si no hay diversidad la población corre un gran riesgo, porque es casi imposible entonces que se pueda adaptar a las condiciones de contexto => **hay que mantener la diversidad en nuestros algoritmos con operadores de variación en la población.**

→ **Población:** población de *potenciales soluciones a algún problema que queramos resolver, de cualquier tipo, que se sepa que tiene solución.*

Algoritmo:

```
Inicializar(Población)
MejorAptitud ← Evaluar(Población)
mientras MejorAptitud < AptitudRequerida
    Progenitores ← SelecciónNatural(Población)
    Población ← ReproducciónVariación(Progenitores)
    MejorAptitud ← Evaluar(Población)
fin
```

Elementos de un algoritmo evolutivo

1. **Representación de los individuos** → poder codificar las soluciones de nuestro problema.
2. **Función de aptitud** → cómo medir qué tan bueno es un individuo para resolver el problema, con un número.
3. **Estrategia de selección** → decir cómo se eligen los padres de la nueva generación, teniendo en cuenta una componente aleatoria.
4. **Operadores de variación** → cómo cruzar el material genético para generar los hijos.
5. **Reproducción y reemplazo generacional** → cómo realizar la reproducción y generar la nueva población.

Algoritmos genéticos: representación de los individuos

¿Cómo traducir el problema práctico que tenemos en esta representación especial que necesitamos para los algoritmos genéticos?

→ **CROMOSOMAS**

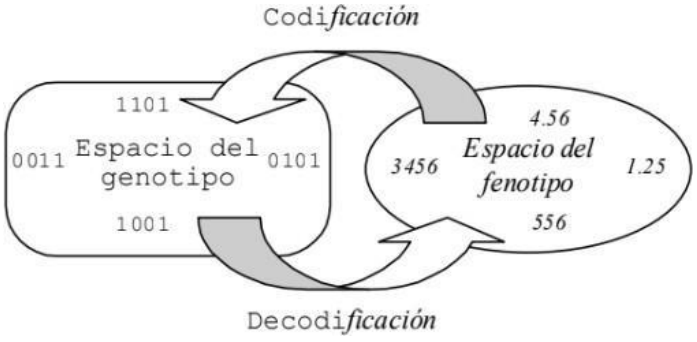
→ que adentro tienen **GENES**

→ que pueden tomar distintos valores llamados **ALELOS**

En cada posición del cromosoma va a haber un gen y ese gen podrá tomar el valor 0 o 1

* → **GENOTIPO**: la información genética que posee un organismo en particular, en forma de ADN. El genotipado se usa para determinar qué variaciones específicas existen en el individuo.
* → **FENOTIPO**: la expresión del genotipo en función de un determinado ambiente. Los rasgos fenotípicos cuentan con rasgos tanto físicos como conductuales

→ **GENOTIPOS informáticos**: dos posibilidades, 0 y 1.
→ **FENOTIPOS informáticos**: problema a resolver, los parámetros/pesos que podemos representar del problema.



→ **Proceso de codificación**: proceso en el cual se convierte de genotipo a fenotipo.

- Lo necesitamos ya que si tenemos el material genético en estado puro, en ceros y unos, no vamos a poder saber si esa es una buena solución o no a nuestro

problema, entonces tenemos que convertirla al dominio del fenotipo donde realmente podemos evaluar qué tan buen solución es para el problema el individuo

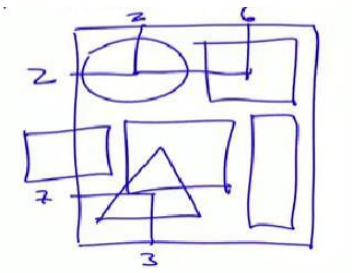
→ **Proceso de decodificación**: proceso inverso. Aunque no es tan crucial, también vamos a tener intenciones de ver, dado una determinada solución del problema, a qué genotipo se corresponde.

→ Vamos a tener muchos individuos, cada uno va a tener sus cromosomas, 1 o más.
→ Cada individuo es una posible solución al problema → codificada.

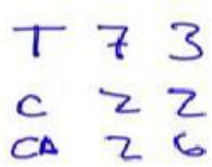
1) Representación de los individuos: ejemplos

- **Ubicación de figuras para el llenado de un área,** aprovechando lo mejor posible el espacio.

Solución: **(pos. figuras)**



Solución codificada:
(fenotipo)



(fig posx posy)

Cromosoma binario: ***(genotipo)***
(3 figuras, 10 bit en total por cada una) [fig posx posy]

[00 0000 0000][01 0000 0000]...
[10 0000 0000]

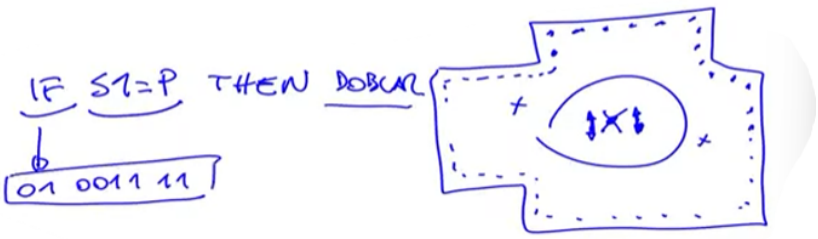
→ Nos queda un individuo de 30 bits de largo → **MATERIAL GENÉTICO DE UN INDIVIDUO**

- **Entrenamiento de una red neuronal**

Fenotipo:
Genotipo:
1- Cant neuronas capa 1 [1enbits 2enbits 3enbits 4enbits ….]
2- Cant neuronas capa 2
3- Cant neuronas
capa 3 (4-N) - pesos

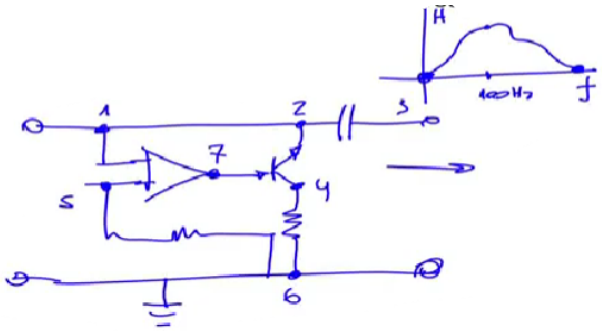
[1 2 3 4 ….. N]

- **Programación de un robot** (debe recorrer el borde de la habitación si chocar las paredes) Vamos a poder construir un programa escrito en código, asignando códigos binarios a las instrucciones

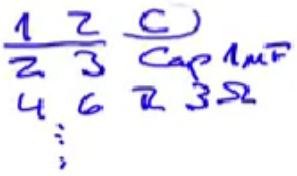


Llegamos a representar en un cromosoma **[instrucción argumentos]**, agregando bits si es necesario → Luego el programa será una serie/cadena de cromosomas

- **Circuito para un filtro multibanda**



FENOTIPO:
Decimos qué hay entre cada nodo:
[N1 N2 cable/capacitor/etc valor]



GENOTIPO: codificación en bits…

- **Problema del agente viajero…(...)**

2) Función de Aptitud (fitness)

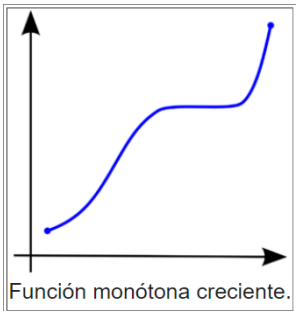
→ Función que mida qué tan bien soluciona el problema un individuo.
→ Nos sirve para elegir a los padres de la próxima generación.
→ Está muy correlacionada con el problema a resolver, pero todas tienen:

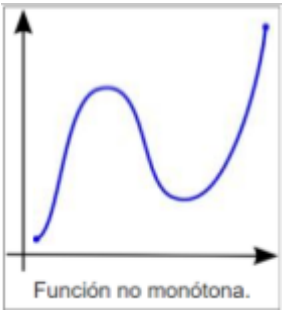
- **Características generales**

➢ ***Monotonicidad***



=>





➢ ***Precisión***

La función debe poder diferenciar individuos que sean parecidos y distinguirlos con números diferentes.

➢ ***Suavidad regulable*** (no es completamente necesaria)

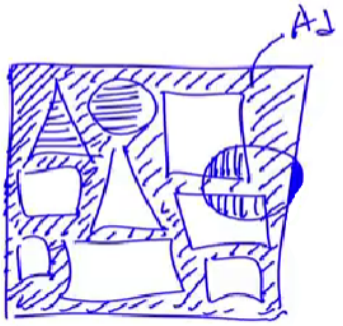
Poder graduar con algún parámetro qué tan escarpada es la forma.
Ej: función sigmoidea → la suavidad del salto la puedo regular con el parámetro alpha.

➢ **Penalización de complejidad**
Ante dos soluciones buenas, que elija la más simple, penalizando la complejidad en la función de aptitud.

- Algunos ejemplos típicos de funciones:
  - Promedios de error: cuadrados medios, desviación media absoluta, error relativo medio,...
  - Estadísticas: estimación de la varianza, validación cruzada, verosimilitud, predicción de error,...
  - Medidas de información: criterio de Akaike, criterio de información Bayesiano, descriptor de mínima longitud, información mutua, minimización del riesgo empírico
  - Otras: correlaciones, distancias,...

Ejemplos de funciones de aptitud para los ejemplos anteriores de individuos

- **Ubicación de figuras para el llenado de un área**



→ Podríamos definirla como el área desperdiciada **Ad** en negativo
=> cuanto menos área halla el valor de aptitud crece.
→ Más el área ocupada **Ao**
→ Menos las áreas solapadas **As**
→ Menos el área que queda afuera **Aa**.

f = - Ad + Ao - As - Aa

→ Podríamos agregarles coeficientes a los términos y darles más o menos pesos.

- **Entrenamiento de una red neuronal**

→ tasa de acierto o error

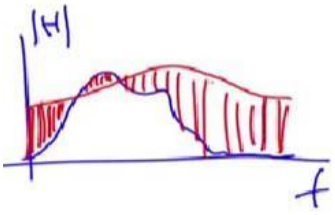
- **Programación de un robot**

→ Definir cerca de las paredes un conjunto de objetivos, y luego contar cuántos de ellos alcanzó, y definir:

f = objetivos_alcanzados - distancia_recorrida

- **Circuito para un filtro multibanda**

→ Medir el error entre la salida del circuito y la esperada



$$f = \frac{1}{1+c}$$

- **Problema del agente viajero**
  → distancia total recorrida + otros criterios de costos asociados a “hotelería” u otros gastos.

*** → Representación de individuos + Función de aptitud se definen MUY en función al problema particular que queremos resolver.

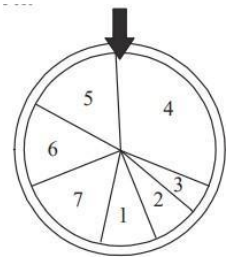
→ una vez hecho esto, lo que resta es más estándar.

Operadores Genéticos

3) Estrategias de selección

→ ***todos los individuos tienen la posibilidad de ser elegidos, pero todos tienen distinta probabilidad****, y esa probabilidad tiene que estar* ***en función de qué tan buenos son.***

a. Método de la ruleta

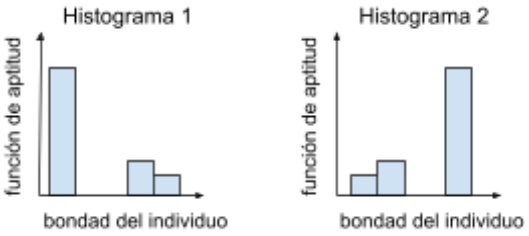


→ cada individuo tiene una porción de la ruleta que es proporcional a su fitness.

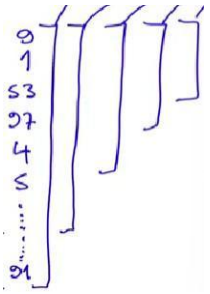
Tiene 2 inconvenientes:

1. Mar de los mediocres (va a caer siempre en los malos porque son muchos más).
2. Mar de los virtuosos (termina eligiendo al azar porque todos tienen una porción similar).

→ en ambos casos hay un *problema con la función de fitness*.



b. Ventanas



→ Utiliza el fitness para ordenar a los individuos, pero descarta el valor absoluto de él.

Primero elige una ventana grande, y saca uno al azar, luego una más pequeña siempre desde arriba hacia abajo, y elige otro,...

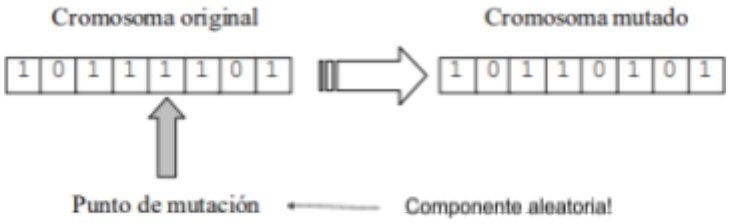
=> de esta manera le estamos dando menos probabilidad de ser elegidos a los últimos que tienen menos fitness, pero la probabilidad no es cero porque en la primera ventana estaban contemplados.

c. **Competencias**

→ Se eligen k individuos cualquiera y el que tenga mayor fitness queda.

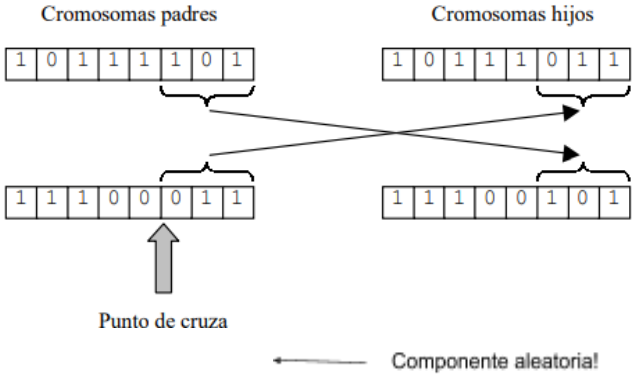
4) Operadores de variación/cruza/mutaciones

Mutación: Elegir un **punto al azar**, y cambiar ese bit:



→ Se definen *tasas de mutación*, con las cuales hay que tener cuidado y especificar *sobre qué se definen*, a nivel de individuos, cromosomas o genes.

Cruzas simples: Elegir un **punto a partir del cual cruzar al azar** e intercambio el material genético.



→ la idea es combinar las partes buenas de 2 padres en un hijo para que sea mucho mejor que los padres.

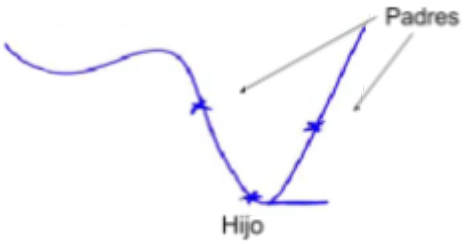
¿Qué rol cumple cada operador en la búsqueda?

Mutación:

★ Sirve para aumentar la diversidad y explorar regiones lejanas del espacio de búsqueda. → tal vez en una zona alejada encontremos el verdadero mínimo global.

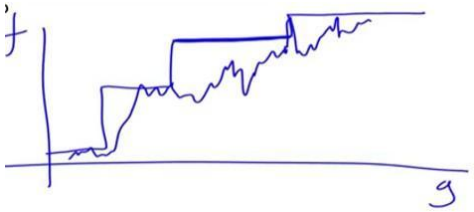
Cruza:

★ Hace una exploración más fina de una zona del espacio.



5) **Reproducción y reemplazo generacional**

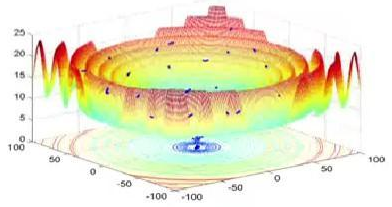
- **Reemplazo total** → Reemplazo la población por todos los hijos que generé.
- **Reemplazo con brecha generacional** → dejamos vivos algunos padres y le sumamos a los nuevos hijos generados
- **Elitismo** → se toma el mejor individuo de la población y *se lo pasa directamente a la próxima generación.*
    - la consecuencia de esto en el fitness en función de las generaciones pega saltos, cuando se encuentra uno mejor, sino, variaría todo el tiempo.

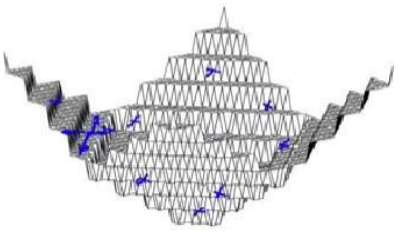


Algoritmos evolutivos: características principales

- ***Búsqueda en un espacio codificado de parámetros***, el genotipo, si bien voy midiendo la función de aptitud sobre el espacio verdadero.
    + Ventaja → en este espacio se asegura la convergencia.

- ***Búsqueda en múltiples puntos del espacio de soluciones*** ←←←←
  Si tenemos 100 individuos, cuando comienza la evolución es como si tomáramos un puñado de semillas y lo tiramos sobre la superficie de error. En cada generación, con cruzas y mutaciones, ellos se van a ir ubicando en otras regiones, y de a poco se van a ir acercando al mínimo global ya que ganan los de mejor fitness, sin quedarse estancados.





- ***Pocos requisitos sobre la función objetivo,*** al contrario del método del gradiente, donde sí necesitamos conocer la derivada y la forma analítica de la función.

- ***Algoritmo de naturaleza estocástica*****,** pero no es completamente aleatorio, combina parte determinística con parte aleatoria.
- ***La estructura de los operadores los hace muy efectivos al realizar búsquedas globales.***
- ***Múltiples objetivos***, pueden perseguir múltiples objetivos a la vez con la función de fitness.

Algunas desventajas:

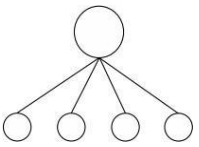
- Son lentos.
- Mucho costo computacional (evaluación de la función de fitness).

Comparación con otros métodos

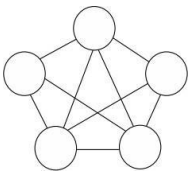
Métodos tradicionales	Algoritmos evolutivos
Trabajan con los propios parámetros a optimizar	Emplea una codificación de los parámetros*
Utilizan información de las derivadas de la función objetivo u otro conocimiento adicional	Utilizan la información de la función objetivo en forma directa
Reglas de transición deterministas	Reglas de transición combinando componentes determinísticos y probabilísticos
Exploran el espacio de soluciones a partir de un punto	Exploran el espacio de soluciones en múltiples puntos a la vez

Paralelismo

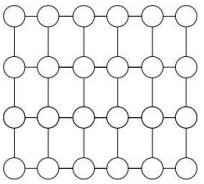
- **Esquema Maestro-Esclavo:**
  - Podemos hacer las cruzas y mutaciones en el servidor, y distribuyó una función de fitness para cada nodo del sistema de paralelismo que tenga con N individuos c/ una.



- **Esquema conexiones:**
  - Formar pequeñas colonias, con N individuos, cada una trabajando en un procesador o grupo de procesadores, y cada tanto se hacen intercambios, todos contra todos, para hacer que evolucione en forma conjunta.



- **Esquemas tipo celdas**
  - Las conexiones no son todos contra todos, sino que se definen regiones donde se intercambia, lo cual requiere cambios en el algoritmo evolutivo en sí mismo.



Computación Evolutiva: Variantes

Parámetros que controlan la evolución

→ Hay distintos parámetros que podrían variar, o modificarse automáticamente: *no ser fijos, o no ser los mismos para todos los individuos, o para distintas partes dentro de un cromosoma*.

- **Probabilidad de mutaciones**
- **Probabilidad de cruzas**
- **Tamaño de la población**
- **Brecha generacional**
- **Elitismo**

  → dan el surgimiento de los algoritmos llamados *"estrategias de evolución"*

Estrategias de evolución

- Representación "**fenotípica**": → poner directamente en los cromosomas los valores de las variables, sin la conversión a binario.
  - Variables objetivo → las que estuvimos evolucionando.
  - Variables de control (o estratégicas) → las que controlan la evolución.
- **Fitness** ... igual que en algoritmos genéticos.
- **Operadores**:
  - Mutación → *lo mutamos y lo guardamos sólo si no empeora.*
  - Cruza (optativa)
- **Selección**: aleatoria, *combinada con mutación.*
- **Reproducción**: determinística, dos mecanismos:
  - ➢ Mecanismo (μ **+** λ)-ES: se generan **μ** padres que producen **λ ≥ 1** hijos.
    → *Próxima generación*: **{μ** ∪ **λ}** eliminando los peores de **λ.** (tam. pobl. cte.)
  - ➢ Mecanismo (μ**,** λ)-ES: se generan **μ** padres que producen **λ > μ** hijos.
    → *Próxima generación*: **subconjunto** con los mejores de **λ.** (tam. pobl. cte.)

Representación de los individuos

Terminología: *algoritmo/programación/estrategia...*

- **Genético** : representación **BINARIA**
  - ➢ Muchos genes con pocos alelos (0 o 1): convergencia asegurada por el teorema de esquemas.
  - ➢ Epistasis: *un gen incorrecto invalida a todo el cromosoma.*
  - ➢ Representación lejana al dominio del problema (ej: viajero con enteros)
  - ➢ Gran cantidad de soluciones inválidas en la población.

- **<u>Evolutivo:</u>** representación **REAL** o "**fenotípica**"
  - ➢ Pocos genes con muchos alelos: representación fenotípica.
  - ➢ Convergencia muy dependiente de los operadores.
  - ➢ *Necesidad de redefinición de operadores no "biológicos".*

Otras representaciones: Cromosomas de longitud variable, Árboles, Grafos…

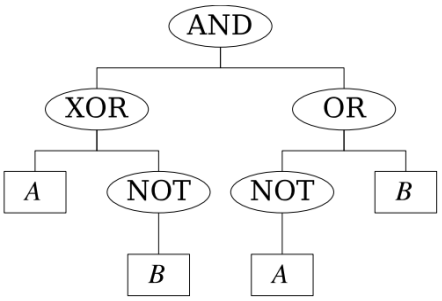
Programación genética

"...generación automática de programas..."

Elementos básicos de un programa:

- Variables y constantes
- Operadores aritméticos y lógicos
- Funciones matemáticas
- Condicionales
- Bucles
- Recursiones
- ...

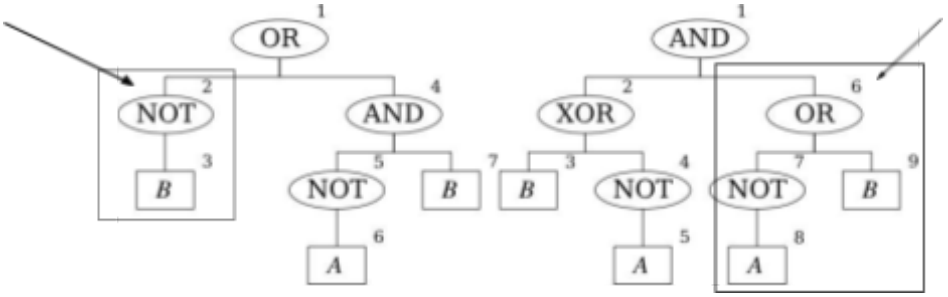
Ejemplo sencillo con operadores lógicos: $((A)\text{XOR}(\text{NOT}(B)))\text{AND}((\text{NOT}(A))\text{OR}(B))$



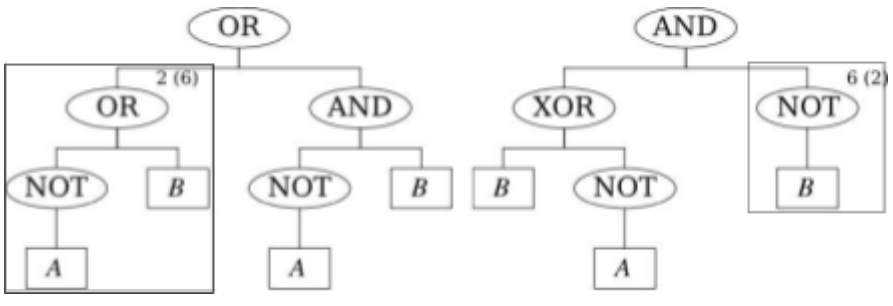
Cruzas en PG

1. <u>Numeración de nodos</u>

   → En este caso primero vamos en profundidad y después vamos recorriendo las ramas
2. <u>Selección de los puntos a cruzar</u> ej 2 y 6



3. <u>Cruza en base al intercambio de ramas</u>



→ Me permitió hacer cruza pero manteniendo la función central de cada árbol, sin hacer cambios tan abruptos.

Mutaciones en PG

- Numeración de nodos
- Selección de la rama a mutar
- Generación de un árbol al azar (con las variables correspondientes)
- Mutación en base al reemplazo
  → el punto clave es que necesitamos una función generadora de árboles al azar.

¿Cómo incorporar en el algoritmo las restricciones del problema real/dominio de la aplicación?

¿Cómo se pueden considerar las restricciones del problema ***durante la evolución***?

- Redefinición de la representación de forma de que **siempre se generen fenotipos válidos.**
- Rechazo o eliminación de individuos inválidos → no es del todo deseable.
- Reparación del material genético.
- **Modificación de los operadores de variación** → para que no hagan individuos inválidos.
- Esquemas de penalización en la función de aptitud.